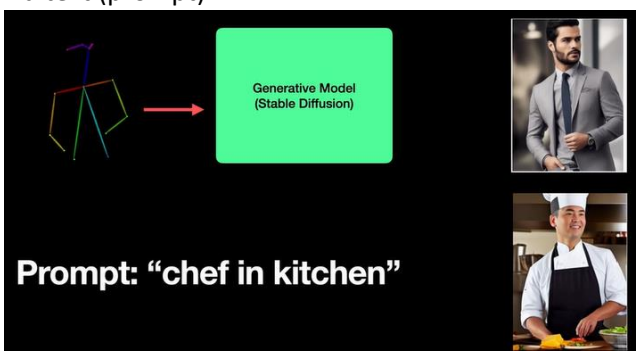
ControlNet

They create high-quality images based on user text prompts, but it can be tough to capture specific spatial details in a text prompt alone.
ControlNet is a end-to-end neural network architecture to add **spatial conditioning controls** for large pretrained text-to-image diffusion models (i.e. Stable Diffusion).

Introduction

When we take image generation models such as Stable Diffusion the quality of the **image generated is mind-blowing** the output however is pretty much controlled by text input (text prompt) but this method is limited in the ability to control the output.
Current text-to-image frameworks offer **restricted control** over spatial composition, making it challenging to specify complex layouts or poses through linguistic descriptions alone. Achieving high-fidelity alignment with a user's intent frequently necessitates an iterative 'trial-and-error' workflow, involving constant prompt refinement and visual verification.
ControlNet allows the users to **add conditions** (i.e. Canny edges, human pose, …) to control the generation of the image of a large pretrained diffusion models.

Example (man standing in a particular pose):
If we want to generate an image of a man standing in a particular pose, we have to spend a lot of time and words to express this idea in a text (prompt).



It would be better if we can feed directly the pose into the model. Moreover, if we pass a short prompt with the initial pose we have a better control on the final output. The problem of precisely controlling the spatial layout of generated images through input images is what is addressed by ControlNet

The inputs could be sketches normal, maps depth, maps edges, segmentation masks or even human pose.

ControlNet:

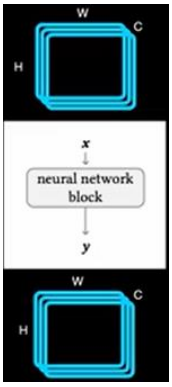
- Add spatially localized input conditions to pretrained diffusion model (text-to-image) via efficient finetuning
- Preserves the quality and capabilities of the pretrained model (locking its parameters).
- The large pretrained model is treated as a backbone for learning diverse conditional controls
- Make a trainable copy of its encoding layers
- The trainable copy and the locked model are connected with zero convolution layers (with weights that progressively grow during the training)
- Training of ControlNet is robust and scalable on different sizes of datasets

If we will finetune (or continue the training) of a large pretrained model with limited data (small dataset) may cause overfitting (and catastrophic forgetting). So we can’t just follow this solution but we have to construct a different process to obtain the goal.

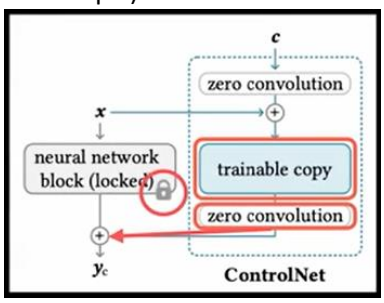
Structure

ControlNet injects additional conditions into neural network block (set of neural layers like ResNet block).
We all know that neuron networks are composed of several neural network blocks. For example, if you take the famous ResNet it has several ResNet blocks in sequence. Similarly, the Transformers has a sequence of multi-headed attention blocks.

To understand the ControlNet architecture let's consider a single block of any neural network from a generative model (i.e. Stable Diffusion). A trained neural block transforms an input feature map *x* into another feature map *y*. It typically takes a three-dimensional tensor with height, width and the number of channels as input and outputs a similar dimensional tensor.



A ControlNet **is added** to such a block. To add control to this block we first make a **copy of the weights** and **freeze** them. This trainable copy is then connected to zero convolutions and the output of zero convolutions is sent back as input to the frozen block. So, whenever we train this model it's this trainable copy that gets updated but the actual pre-train generated model remains frozen. The trainable copy takes a conditioning vector *c* as input (along with input feature map *x*).



The complete output of the ControlNet is computed as follows:

$$y_c = \mathcal{F}(x;\Theta) + \mathcal{Z}(\mathcal{F}(x + \mathcal{Z}(c;\Theta_{z1});\Theta_c);\Theta_{z2})$$

Where:

- **F(x; Θ)** is the output of the original block with parameters **Θ**.
- **Z(c; Θ_{z1})** is the output of the first zero convolution layer.
- **F(x + Z(c; Θ_{z1}); Θ_c)** represents the output of trainable copy model
- **Z(F(x + Z(c; Θ_{z1}); Θ_c); Θ_{z2})** is the output of the trainable copy after the zero convolution layer
- **c** is the control hint (like depth map)

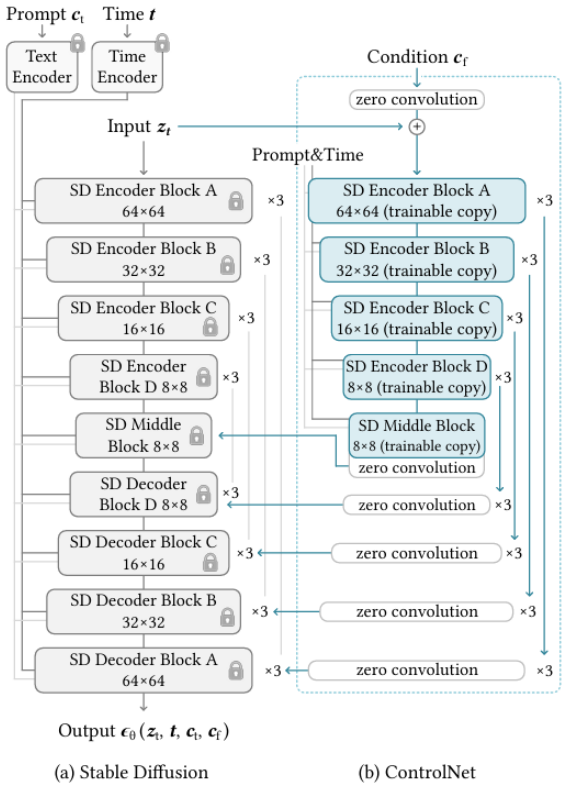
ControlNet for Stable Diffusion

Scaling up this idea to Stable Diffusion which has encoder and decoder layers, the blocks in blue are the trainable copies of the encoder layers. The output of each of these block is put through the zero convolutions which are fed into the stable diffusion decoder blocks. One more place where zero convolutions are used is when we take the input conditioning such as the depth map or the pose and combine them with the input representation z_t.

The *locked parameters* preserve the production model while the *trainable copy* reuses such pretrained model to establish the backbone for handling diverse input conditions.

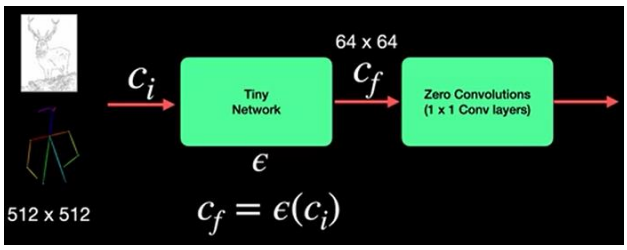
Zero convolutions these are simple 1x1 convolution layers whose weights and biases are initialized to zero. Normally, when you add a new layer to a neural network, you initialize it with **random numbers**. If you did this with ControlNet, the first time you ran the model, it would inject random "noise" into the pretrained Stable Diffusion model.
These layers protect the backbone by eliminating random noise as gradients in the initial training steps. So in this way, harmful noise cannot influence the hidden states of the neural network layers in the trainable copy when the training starts. This approach results in a smooth integration where ControlNet's influence on image generation progressively increases as training continues.
In the beginning the trainable copy is fully functional and retains the capabilities of the large, pretrained model (notice that receive the input image *x*). In other words, at the start of the ControlNet training the output is the same as the output of the original block

Stable Difffusion architecture has 12 encoders and decoders with a middle block



Stable Diffusion doesn't work with full-sized images. Instead, it works within a compressed, simplified version of the image data. Instead of training on massive 512x512 pixel images Stable Diffusion operates in a **latent space**. To get into this latent space, the model uses a pre-processing technique (**VQ-GAN**). This shrinks a standard 512 x 512 image down into a compact 64x64 latent images. For **ControlNet** to work, it has to "speak

the same language" as Stable Diffusion. We first convert each input conditioning image (e.g., edge, pose, depth, etc.) from an input size of 512 ×512 into a 64×64 feature space vector that matches the size of Stable Diffusion.
In particular, we use a tiny network $\epsilon(\cdot)$ of four convolution layers with 4 × 4 kernels and 2 ×2strides to encode an image-space condition c_i into a feature space conditioning vector c_f.



The conditioning vector c_f is passed into the ControlNet.

Training

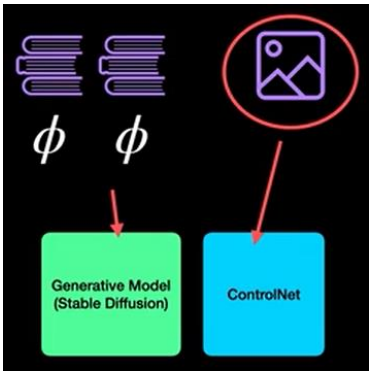
Given an input image z_0, the diffusion algorithm *progressively add noise* and produce z_t (noisy image). The diffusion algorithm learns a network ϵ_θ to predict the noise added to z_t using the following **overall learning objective** of the entire diffusion model:

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{z}_0, \boldsymbol{t}, \boldsymbol{c}_t, \boldsymbol{c}_\mathrm{f}, \epsilon \sim \mathcal{N}(0,1)} \left[\| \epsilon - \epsilon_\theta(\boldsymbol{z}_t, \boldsymbol{t}, \boldsymbol{c}_t, \boldsymbol{c}_\mathrm{f}) \|_2^2 \right]$$

t: current time step (number of times noise is added)
z_0: input image
z_t: noisy image that is produced
c_t: text prompt guiding the image generation
c_f: task-specific condition
ϵ: actual noise added
ϵ_θ: model's guess about the noise added to the noisy image z_t

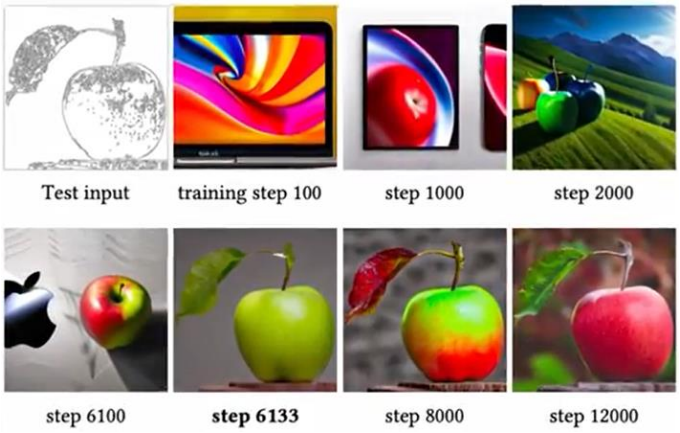
The loss (***L***) is the difference between the real noise and the predicted noise. The goal of training is to make this difference as small as possible.

We would like to introduce image prompts instead of text prompts to shift the control from text to image prompts so we deliberately replace half the text prompts (c_t) in the training data with empty strings so that network is now **forced to learn the semantics** of the conditioning images such as the edges or the poses or the depth (as a replacement for the prompt)



Training this way the model abruptly succeeds in following the input conditioning image usually in less than 10,000 optimization steps and this phenomenon is referred to as the "**sudden convergence phenomenon**".

Example (apple):
We can notice that around step 6,100 though the output is an apple the spatial layout of this apple is quite different from that of the input edge conditioning in the test image. At 12,000 steps it's only going to get better at matching the semantics of the input image prompt



Due to the zero convolutions, ControlNet always predicts high-quality images during the training because they don't add noise to the network. At a certain step in the training process (e.g., 6133 steps), the model suddenly learns to follow the input condition.

Inference

If we move on to inference ControlNet introduces what is called "**classifier-free guidance resolution weighting**". Let's break it down and start from classifier guidance then move on to classifier free guidance and then resolution weighting.

We often want **to control the generated images**, such as generating only dogs or cats. In such cases, we need to introduce conditional controls. Classifier-free guidance and classifier guidance are two techniques used to steer the output of generative models, like diffusion models, toward desired attributes (e.g., generating images of a specific class).

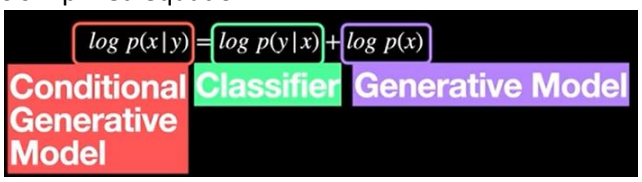
Classifier guidance

Let's get begin with the Bayes theorem by simply applying log to this equation we can convert multiplications to additions and divisions to subtractions so the modified equation becomes like this:

$$\log p(x|y) = \log p(y|x) + \log p(x) - \log p(y)$$

Bayes theorem after the conversion

Lastly assuming that the distribution of the labels p(y) is trivial or if we take them for granted we can set it to zero and so we arrive at this simplified equation:



log p(x)*: is simply the distribution of the input data x. It's a generative model because if we simply sample from this distribution of x we can get outputs similar to that of x.*

log p(y|x)*: it represents a discriminative classifier (given an input x give me the probability of class y). It depends on the prompt*
log p(x|y)*: is the conditional generative model (given a label y give me a generated sample x from the distribution)*

With this approach we have to **double the workload** because we have to take care of two models: train the classifier and the model.

Classifier Guidance **can only control the categories generated by the classification model**. If the classification model distinguishes 10 classes, then Classifier Guidance can only guide the diffusion model to generate those fixed 10 classes; it won't work for an additional class.

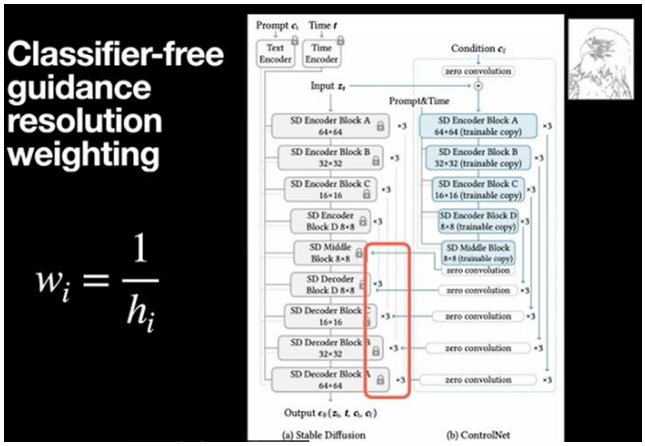
By simply adding a classifier to the generative model we get a conditional generative model. This is the exact idea of classifier guidance but another of the drawback of this classifier is that it **needs to be trained with noisy input** because the input to the diffusion model is noise and so the classifier needs to take the noise and classify what class that noise belongs to. If the classifier isn't robust to noise or misaligned with the generative model, **guidance may fail**. The solution to this problem however is to get rid of the classifier all together and arrive at a classifier free guidance.

Classifier-free guidance (CFG)

The next step is to **get rid of the classifier**. Classifier-free guidance is an alternative method to have the same effect as classifier guidance, but without a classifier. It can work even when the condition is text or image. We don't need to train a classifier, we need a conditional generative model and an unconditional generative model. CFG enhances the quality and condition adherence of text-to-image diffusion models. It operates by combining the conditional and unconditional predictions using a fixed weight. Rather than sampling in the direction of the gradient of an image classifier, classifier-free guidance instead mixes the score estimates of a *conditional diffusion model* and a jointly trained *unconditional diffusion model*.

Classifier-free guidance resolution weighting

In ControlNet architecture we have two networks the diffusion model and its train copy (the ControlNet) whenever we have a conditioning image (i.e. an edge map) we introduce additional weights that are multiplied at the connections between the diffusion model and the ControlNet so that the output is much more refined. Now these weights are not neural network weights but simple numbers which are multiplied at each stage. The weights are inversely proportional to the size of the blocks we are connecting into.



For example when the block size is 8x8 the weight is 1x8 and if the block size is 64 then the weight is 1x64

Clearly the results indicate that by using this resolution waiting shown by the image on the right as full without prompt we are getting best results compared to being generated without the weighted guidance

(a) Input Canny map (b) W/o CFG (c) W/o CFG-RW (d) Full (w/o prompt)

Figure 5: Effect of Classifier-Free Guidance (CFG) and the proposed CFG Resolution Weighting (CFG-RW).

Ablation studies

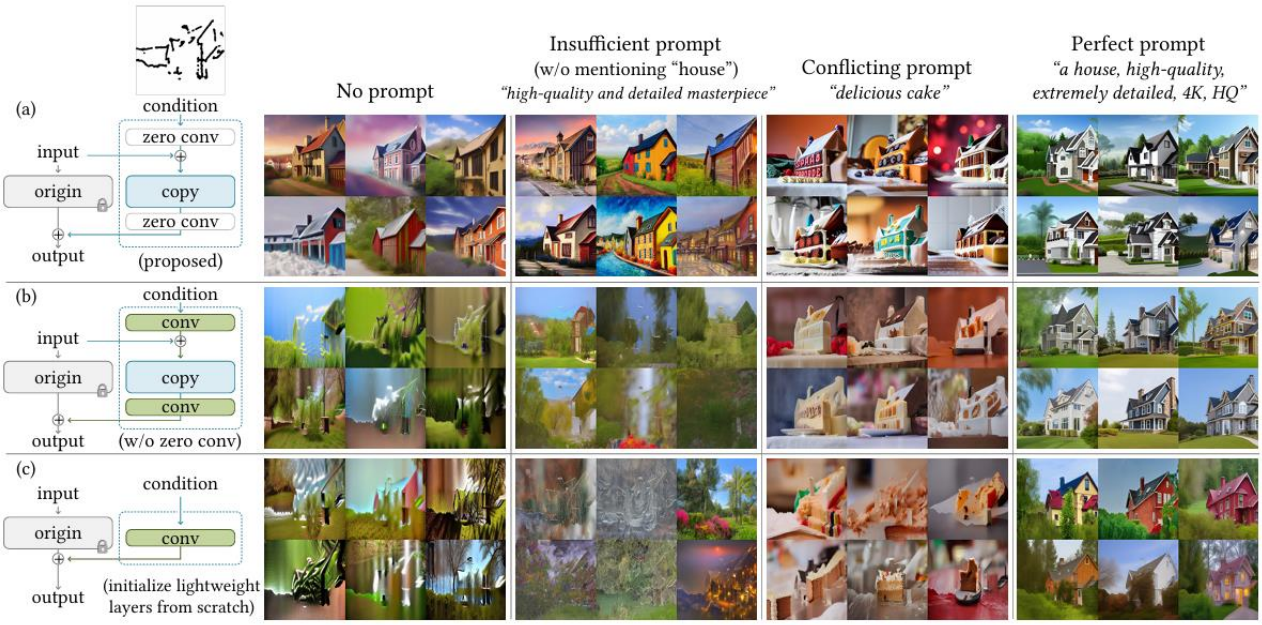


Figure 8: Ablative study of different architectures on a sketch condition and different prompt settings. For each setting, we show a random batch of 6 samples without cherry-picking. Images are at 512 × 512 and best viewed when zoomed in. The green "conv" blocks on the left are standard convolution layers initialized with Gaussian weights.

The ablations study is organised exploring different configurations to observe how the architecture reacts.

Architecture settings:

a) **Full ControlNet**: use original ControlNet
b) **ControlNet without zero convolutions**: use standard convolutional layers (replacing the zero convolutions).
c) **Lightweight ControlNet**: replace trainable copy with one convolution layer (ControlNet-lite)

Prompt settings:

1) No prompt
2) Insufficient prompt
3) Conflicting prompt
4) Perfect prompt

Results:

a) ControlNet succeeds in all 4 settings
b) The performance of ControlNet drops under insufficient and no prompt indicating that the pretrained backbone of the trainable copy is destroyed during the finetuing. This is an evidence that zero-initialized connections protect the pretrained copy during finetuning.
c) The lightweight ControlNet isn't strong enough to interpret the conditioning images failing with no prompts and insufficient prompt.

More studies

They also study the data set size that is needed to train ControlNet and show that even though we can get reasonable outputs with just 1000 images we can see that with a lot more images say 50,000 the results look better and as we get closer to 3 million it gets even better.

Lastly ControlNet is not just restricted to a Stable Diffusion it can also be applied to other generative models like Comic Diffusion and Protogen so to wrap things up these are the variety of control images you can provide as input along with the generated images for each of them. it includes sketches, normal Maps, depth maps, edges and human pose.